Lộ trình 6 tuần tiếp theo (Week 3 – Week 8)

Đề tài: Dự báo hạn hán bằng SPEI và cảnh báo nguy cơ mặn kèm hệ khuyến nghị RAG cho Đồng bằng sông Cửu Long

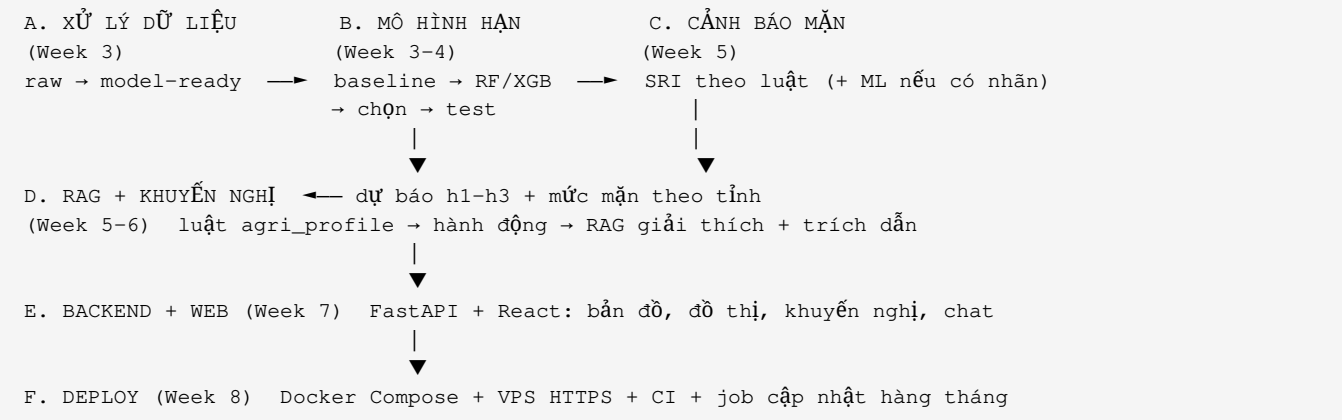
Ngày lập: 22/10/2026 | **Giai đoạn:** 26/10/2026 – 06/12/2026

Hiện trạng (hết Week 2): dữ liệu ERA5 13 trạm × 10 biến (1991–2025, 0 lỗi), ONI, DMI, lưu lượng Mekong; `dataset_monthly_ml.csv` gồm 4446 dòng × 94 cột; SPEI đạt chuẩn và khớp các đợt hạn mặn 2016, 2020, 2024. Còn thiếu: nhãn độ mặn, kho tài liệu RAG, mô hình, web.

> **Cách dùng tài liệu này:** đọc theo thứ tự các giai đoạn A → F. Mỗi bước ghi rõ **Mục tiêu – Đầu vào – Việc cần làm – Đầu ra – Kiểm tra**. **Giai đoạn A có code đầy đủ** để chép vào project sau khi kiểm tra (chưa chạy thử); các giai đoạn B–F hiện chỉ mô tả cách làm.
>
> **Nguyên tắc chung:** không sửa các module đã chạy ổn (`mekong.config`, `ingestion`, `quality`, `features`). Chỉ **thêm** module mới vào các package có sẵn (`forecasting`, `salinity`, `rag`, `storage`, `services`, `api`, `web`). Mọi tham số đặt trong `config.yaml`, mọi đường dẫn lấy qua `mekong.config.get_path`, mọi script chạy được bằng `python -m mekong.<module>` và có target tương ứng trong `Makefile`.

---

0. Luồng tổng thể



Giai đoạn	Tuần	Điều kiện bắt đầu	Kết quả bàn giao
A. Xử lý dữ liệu	3	Dataset Week 2 (đã có)	`model_input_h{1,2,3}.csv`, danh sách đặc trưng, test pass
B. Mô hình hạn	3–4	Xong A	3 mô hình (h1–h3) đạt KPI, `models/`, báo cáo
C. Cảnh báo mặn	5	Xong B (cần SPEI dự báo)	SRI 13 tỉnh, (mô hình mặn nếu đủ nhãn)
D. RAG + khuyến nghị	5–6	Kho tài liệu ≥ 60 file; xong B, C	Khuyến nghị có trích dẫn, đánh giá RAG
E. Backend + Web	7	Xong B, C, D	Web chạy local đủ 3 luồng
F. Deploy	8	Xong E	URL public, CI, job hàng tháng, báo cáo

Việc làm **song song suốt Week 3–5**: thu thập nhãn mặn và tài liệu cho RAG (mục C1, D1).

---

1. Endgame và tiêu chí nghiệm thu

#	Yêu cầu	Tiêu chí đo được (Definition of Done)

E1	Tỉ lệ dự đoán đúng > 60%	**Balanced accuracy ≥ 0,60** cho phân loại 3 mức hạn (SPEI-3) ở horizon 1 và 2 tháng trên tập test 2022–2025, **và recall lớp "Hạn" ≥ 0,60** ở horizon 1. Horizon 3 tháng: vượt cả persistence lẫn climatology
E2	Giao diện trực quan	Bản đồ 13 tỉnh tô màu theo mức hạn/mặn dự báo (chuyển được địa giới cũ / sau sáp nhập 2025); tối thiểu 6 loại đồ thị dựng từ đầu ra mô hình
E3	Website chạy được và deploy được	`docker compose up` dựng đủ DB + API + web; có bản public qua HTTPS; CI chạy test tự động
E4	RAG + hệ khuyến nghị	Khuyến nghị theo tỉnh, cây trồng/vật nuôi và tháng, mỗi khuyến nghị có trích dẫn nguồn; bộ 40 câu hỏi kiểm thử đạt Hit@5 ≥ 0,8 và ≥ 90% câu trả lời có trích dẫn đúng
E5	Chỉ rõ thuật toán ML	Danh sách cố định ở mục 4.1. Mọi mô hình đánh giá trên cùng cách chia tập và cùng bộ chỉ số

---

2. Định nghĩa "tỉ lệ dự đoán đúng" và cơ sở khả thi

2.1. Vì sao không dùng accuracy thường

Nhãn 3 lớp theo SPEI-3 của tháng t+h: **Hạn** (SPEI ≤ −1), **Bình thường** (−1 < SPEI < 1), **Ẩm** (SPEI ≥ 1). Lớp "Bình thường" chiếm khoảng 66% số mẫu. Một mô hình luôn đoán "Bình thường" (climatology) đã có accuracy **0,67** mà không phát hiện được tháng hạn nào.

Vì vậy dùng **balanced accuracy** làm chỉ tiêu chính: trung bình recall của 3 lớp, ngưỡng ngẫu nhiên là 0,33. Đi kèm là **recall và F1 của lớp Hạn**. Accuracy thường, macro-F1, RMSE/MAE của giá trị SPEI và skill score so với persistence vẫn được báo cáo đầy đủ.

2.2. Thử nghiệm khả thi (22/10/2026)

Thiết lập: train ≤ 2018, đánh giá trên val + test (2019–2025, 1014 mẫu), chưa tinh chỉnh tham số.

Horizon	Mô hình	Accuracy	Balanced acc.	F1 lớp Hạn	Recall lớp Hạn
1 tháng	Climatology	0,674	0,333	0,000	0,000
1 tháng	Persistence	0,677	0,598	0,600	0,600
1 tháng	Logistic Regression	0,686	0,673	0,595	0,615
1 tháng	Random Forest	0,738	0,636	0,669	0,600
1 tháng	Gradient Boosting (cây)	0,725	**0,690**	0,643	**0,681**
2 tháng	Persistence	0,640	0,537	0,430	0,430
2 tháng	Gradient Boosting (cây)	0,623	**0,614**	0,507	0,563
3 tháng	Persistence	0,554	0,423	0,326	0,326
3 tháng	Gradient Boosting (cây)	0,529	0,475	0,392	0,415

Ghi chú quan trọng:

- Thử nghiệm này đã đưa cả cột `oni` và `dmi` **của tháng hiện tại** vào làm đặc trưng. Đây là **rò rỉ thông tin** (bước A5), nên con số có thể hơi lạc quan. Bước A11 sẽ chạy lại với bộ đặc trưng sạch và thay bảng này.
- Đánh giá chính thức chỉ dùng **test 2022–2025**. Tập test chỉ có **42 mẫu lớp Hạn**, nên recall lớp Hạn dao động khoảng ±0,15. Cần báo cáo khoảng tin cậy bằng bootstrap (bước B5).
- **Kết luận:** horizon 1 tháng gần như chắc chắn đạt 60%; horizon 2 tháng sát mốc, cần tinh chỉnh; horizon 3 tháng chỉ cam kết vượt baseline.

---

3. Giai đoạn A — Xử lý dữ liệu (Week 3, làm đầu tiên)

> **Trạng thái code trong mục này:** code được viết sẵn để bạn chép vào project khi đã kiểm tra xong. **Code chưa được chạy thử** trên project. Các con số "kỳ vọng" về số cột sau biến đổi

(91 và 73) là tính tay từ danh sách cột; chạy lần đầu cần đối chiếu.

Phong cách code theo các module hiện có:

- dùng `load_config`, `get_path`, `setup_logging` từ `mekong.config`;
- chạy bằng `python -m ...`; có `argparse` với `--config`.

Các file sẽ tạo mới:

- `src/mekong/quality/freeze.py`
- `src/mekong/forecasting/transformers.py`
- `src/mekong/forecasting/preprocess.py`
- `src/mekong/forecasting/evaluate.py`
- `src/mekong/forecasting/baseline.py`
- `tests/test_preprocess.py`

Các file sẽ sửa: thêm mục `forecasting:` vào cuối `config.yaml`; thêm 4 target vào `Makefile`.

A0. Chuẩn bị môi trường làm việc

- **Mục tiêu:** theo dõi được mọi thay đổi trước khi thêm module mới.
- **Việc cần làm:** project hiện chưa phải git repository. `.gitignore` đã loại `data/raw/`, `.venv/`, `.env`.

```
cd /Users/maiphutan/mekong_drought_forecast_RAG
git init
git add .
git commit -m "Week 1-2: pipeline du lieu 13 tram, dataset thang"
git checkout -b week3-preprocess
make test          # 5 test hien co phai pass truoc khi lam tiep
```

- **Lưu ý:** `data/processed/` (khoảng 12 MB) sẽ được commit. Nếu không muốn, thêm `data/processed/*.csv` vào `.gitignore` trước khi chạy `git add`.

A1. Khóa dữ liệu thô (tái lập được)

- **Mục tiêu:** chứng minh được dữ liệu thô không bị sửa sau khi tải.
- **Đầu vào:**
  - `data/raw/*_daily_era5_*.csv` (91 file);
  - 3 file tải về trong `data/external/`: `oni.ascii.txt`, `dmi.had.long.data`, `discharge_daily.csv`.
  - Không đưa `agri_profile.yaml` và template nhãn mặn vào manifest, vì đây là file nhóm tự sửa.
- **Đầu ra:** `data/MANIFEST.sha256` gồm 94 dòng. Manifest đặt ở `data/` chứ không ở `data/raw/`, vì `data/raw/` bị `.gitignore` loại; manifest cần được commit.
- **Tách file cũ:** việc chuyển 104 file `*_soil_era5_*` sang `data/raw/_legacy_soil/` là tùy bạn, làm tay được. Glob `*_daily_*` trong code không đụng tới các file này.

```
mkdir -p data/raw/_legacy_soil && mv data/raw/*_soil_era5_*.csv data/raw/_legacy_soil/
```

File mới `src/mekong/quality/freeze.py`:

```
"""Khóa dữ liệu thô bằng SHA-256 để chứng minh dữ liệu không bị sửa sau khi tải.

Chạy:  python -m mekong.quality.freeze           # ghi data/MANIFEST.sha256
       python -m mekong.quality.freeze --check   # so lại, thoát 1 nếu có file bị đổi/mất
Sau khi chủ động tải lại dữ liệu (make fetch / make external) thì chạy lại lệnh ghi.
"""
import argparse
import hashlib
import sys

from mekong.config import ROOT, get_path, load_config, rpath
```

```
MANIFEST = "data/MANIFEST.sha256"
EXTERNAL_FILES = ["oni.ascii.txt", "dmi.had.long.data", "discharge_daily.csv"]


def file_sha256(path, chunk=1 << 20):
    h = hashlib.sha256()
    with open(path, "rb") as f:
        for block in iter(lambda: f.read(chunk), b""):
            h.update(block)
    return h.hexdigest()


def tracked_files(cfg):
    """Các file thô cần khóa: khối ERA5 theo ngày + dữ liệu ngoài tải về."""
    ext = get_path(cfg, "external")
    files = list(get_path(cfg, "raw").glob("*_daily_*.csv"))
    files += [ext / name for name in EXTERNAL_FILES if (ext / name).exists()]
    return sorted(files, key=lambda p: p.relative_to(ROOT).as_posix())


def build_manifest(cfg):
    return {p.relative_to(ROOT).as_posix(): file_sha256(p) for p in tracked_files(cfg)}


def read_manifest(path):
    out = {}
    for line in path.read_text(encoding="utf-8").splitlines():
        if line.strip():
            digest, rel = line.split(maxsplit=1)
            out[rel.strip()] = digest
    return out


def main(argv=None):
    ap = argparse.ArgumentParser(description=__doc__.splitlines()[0])
    ap.add_argument("--config", default=None)
    ap.add_argument("--check", action="store_true", help="so với manifest đã ghi")
    args = ap.parse_args(argv)

    cfg = load_config(args.config)
    path = rpath(MANIFEST)
    current = build_manifest(cfg)

    if not args.check:
        path.write_text("".join(f"{d}  {p}\n" for p, d in current.items()), encoding="utf-8")
        print(f"đã ghi {len(current)} file vào {path}")
        return

    if not path.exists():
        print(f"chưa có {path}: chạy 'make freeze-raw' trước")
        sys.exit(1)
    old = read_manifest(path)
    changed = [p for p in old if p in current and current[p] != old[p]]
    missing = [p for p in old if p not in current]
    added = [p for p in current if p not in old]
    for label, items in (("ĐỔI", changed), ("MẤT", missing), ("MỚI", added)):
        for p in items:
            print(f"[{label}] {p}")
    print(f"\n{len(old)} file trong manifest: {len(changed)} đổi, {len(missing)} mất, {len(added)} mới")
    sys.exit(1 if changed or missing else 0)


if __name__ == "__main__":
    main()
```

Thêm vào `Makefile`. Sửa dòng `.PHONY` và thêm 2 target. Lưu ý: dòng lệnh dưới mỗi target phải bắt đầu bằng **TAB**, không phải dấu cách.

```
.PHONY: install fetch external validate build data test freeze-raw check-raw preprocess baseline
```

```
freeze-raw: ## ghi checksum dữ liệu thô
    $(RUN) mekong.quality.freeze

check-raw:  ## kiểm tra dữ liệu thô không bị sửa
    $(RUN) mekong.quality.freeze --check
```

- **Kiểm tra:**

  1. `make freeze-raw` báo "đã ghi 94 file".
  2. `make check-raw` thoát mã 0.
  3. Sửa thử 1 ký tự trong một file của `data/raw/` → `make check-raw` phải báo `[ĐỔI]` và thoát mã 1. Hoàn tác lại bằng cách tải lại file đó.

- **Quy tắc từ nay:** không sửa tay file trong `data/raw/`.

A2. Dựng lại pipeline hiện có và ghi lại số liệu gốc

- **Mục tiêu:** đảm bảo đầu vào của giai đoạn A đúng như Week 2. Bước này không cần code mới.

```
make check-raw
make validate      # kỳ vọng: 0 ERROR, 8 WARN
make build         # kỳ vọng: full (5460, 94), ml (4446, 94)
```

Kết quả phải khớp:

Hạng mục	Giá trị
`dataset_monthly_ml.csv`	4446 × 94, 0 NaN; 13 trạm × 342 tháng (1997-04 → 2025-09)
Chia tập	train 3354, val 429, test 585, gap 78

Nếu không khớp: dừng lại, xem `reports/validation_report.csv`.

A3. Khai báo tham số xử lý trong `config.yaml`

- **Mục tiêu:** mọi quyết định xử lý nằm trong config, giống cách mục `features:` đang làm.
- **Việc cần làm:** thêm đoạn sau vào **cuối** `config.yaml`, giữ nguyên các mục khác.

```
# --- Tiền xử lý & mô hình dự báo hạn (mekong.forecasting) ---
forecasting:
  horizons: [1, 2, 3]
  class_thresholds: [-1.0, 1.0]        # Hạn <= -1 < Bình thường < 1 <= Ẩm
  class_names: [drought, normal, wet]  # mã lớp 0, 1, 2
  id_cols: [date, station, split]
  drop_cols: [province, province_2025, lat, lon, month, et0_hg_sum]
  leak_cols: [oni, dmi]                # chỉ dùng oni_lag2/3, dmi_lag1
  anomaly_cols: [sm_0_7, sm_7_28, sm_28_100]          # tự lấy thêm các cột _lag
  log1p_cols: [hot_days, dry_spell_max]
  log_cols: [q_KR_mean, q_KR_min, q_TC_mean, q_TC_min]
  linear_drop: [precip_sum, rain_days, q_KR_mean, q_KR_min, q_KR_anom,
                sm_7_28, easterly_days, tmax_max, q_TC_mean]   # tự lấy thêm các cột _lag
  cv_test_years: [2010, 2011, 2012, 2013, 2014, 2015, 2016, 2017, 2018]
  cv_gap_months: 3
  random_state: 42
```

- **Kiểm tra:** test `test_forecasting_config` ở bước A10.

A4. Tạo nhãn

- **Mục tiêu:** có nhãn hồi quy, nhãn 3 lớp và nhãn nhị phân "Hạn" cho mỗi horizon.
- **Việc cần làm:** đặt các hàm `to_class` và `make_labels` trong `src/mekong/forecasting/preprocess.py`. **Toàn văn file nằm ở bước A10**; đoạn dưới chỉ là phần liên quan.

```
def to_class(spei, thresholds):
```

```
    """SPEI -> 0 Hạn (<= lo), 1 Bình thường, 2 Ẩm (>= hi)."""
    lo, hi = thresholds
    s = np.asarray(spei, dtype=float)
    return np.select([s <= lo, s >= hi], [0, 2], default=1)


def make_labels(df, h, thresholds, scale=3):
    y = df[f"target_spei{scale}_h{h}"]
    return pd.DataFrame({
        "y_reg": y,
        "y_cls": to_class(y, thresholds),
        "y_drought": df[f"target_drought_h{h}"].astype(int),
    }, index=df.index)
```

Số lượng phải khớp (Hạn / Bình thường / Ẩm):

Horizon	train	val	test
h1	579 / 2005 / 770	93 / 308 / 28	42 / 375 / 168
h2	579 / 2005 / 770	93 / 296 / 40	42 / 371 / 172
h3	567 / 2017 / 770	93 / 284 / 52	42 / 376 / 167

- **Lưu ý:**
  - Val chỉ có 28 mẫu lớp Ẩm ở h1, nên khi chọn mô hình cần xem thêm điểm kiểm định chéo (A9).
  - Test (2022–2025) ẩm hơn train. Đây là dịch chuyển phân bố thật, cần nêu trong báo cáo.

A5. Chọn cột đặc trưng và chống rò rỉ

- **Mục tiêu:** ma trận đặc trưng chỉ chứa thông tin đã biết vào cuối tháng t.
- **Lý do loại từng nhóm cột:**

Cột	Lý do loại
`date`, `split`, `province`, `province_2025`	Định danh / metadata
`lat`, `lon`	Trùng thông tin với `station`
`month`	Đã có `month_sin`, `month_cos`
`et0_hg_sum`	Chỉ dùng kiểm tra chéo ET0; tương quan 0,92 với `tmax_mean`
`oni`	ONI tháng t là trung bình t−1..t+1 → cuối tháng t chưa biết → **rò rỉ**
`dmi`	NOAA công bố trễ khoảng 1 tháng → **rò rỉ**

- **Code** (trong `preprocess.py`):
  - `feature_columns` trả về 77 tên cột số.
  - `select_features` trả thêm `date` và `station`, vì bước A6 cần hai cột này. Chúng **không** phải đặc trưng và sẽ bị bỏ ở cuối pipeline.

```
def feature_columns(df, fc):
    """77 cột số được phép làm đặc trưng (đã loại định danh, metadata, rò rỉ, nhãn)."""
    banned = set(fc["id_cols"]) | set(fc["drop_cols"]) | set(fc["leak_cols"])
    return [c for c in df.columns if c not in banned and not c.startswith(("target_", "y_"))]


def select_features(df, fc):
    """Đầu vào của pipeline: date + station (dùng nội bộ) + 77 cột số."""
    return df[["date", "station", *feature_columns(df, fc)]]
```

- **Kiểm tra:** `len(feature_columns(df, fc)) == 77`; không chứa `oni`, `dmi`, `date`, `split`, `target_*` (test A10).

A6. Biến đổi đặc trưng (transformer của scikit-learn)

- **Mục tiêu:** xử lý độ lệch giữa các trạm (ô lưới BL) và phân bố lệch, theo cách fit trên train rồi áp lại cho val, test và từng fold kiểm định chéo.
- **Ý chính:**
    - `StationMonthAnomaly` tính mean/std theo (trạm, tháng trong năm) **chỉ trên dữ liệu được fit**.
    - Với cột `_lagk`, tháng tham chiếu là **t−k** (vì giá trị đó thuộc tháng t−k).
    - Tên cột giữ nguyên, nhưng sau biến đổi `sm_0_7` mang nghĩa **dị thường**.

File mới `src/mekong/forecasting/transformers.py`:

```
"""Bộ biến đổi đặc trưng theo chuẩn scikit-learn.

Mọi lớp nhận và trả về DataFrame; đặt trong Pipeline để fit lại ở từng fold (chống rò rỉ).
"""
import logging
import re

import numpy as np
import pandas as pd
from sklearn.base import BaseEstimator, TransformerMixin

log = logging.getLogger("forecasting.transformers")


def lag_of(col):
    """'sm_0_7_lag2' -> 2, 'sm_0_7' -> 0."""
    m = re.search(r"_lag(\d+)$", col)
    return int(m.group(1)) if m else 0


def expand_with_lags(cols, columns):
    """Danh sách cols + các cột <c>_lagk tương ứng có trong `columns` (giữ thứ tự, không trùng)."""
    out = []
    for c in cols:
        pat = re.compile(rf"{re.escape(c)}(_lag\d+)?")
        out += [x for x in columns if pat.fullmatch(x) and x not in out]
    return out


class HotDayFlag(BaseEstimator, TransformerMixin):
    """Thêm cờ has_hot_day: 88% số tháng có hot_days = 0."""

    def __init__(self, col="hot_days"):
        self.col = col

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X = X.copy()
        X["has_hot_day"] = (X[self.col] > 0).astype(int)
        return X


class LogFeatures(BaseEstimator, TransformerMixin):
    """log1p cho cột đếm lệch phải, log cho lưu lượng (luôn > 0)."""

    def __init__(self, log1p_cols=(), log_cols=()):
        self.log1p_cols = log1p_cols
        self.log_cols = log_cols

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X = X.copy()
        for c in self.log1p_cols:
            if c in X:
                X[c] = np.log1p(X[c].clip(lower=0))
```

```
        for c in self.log_cols:
            if c in X:
                X[c] = np.log(X[c].clip(lower=1e-6))
        return X


class DropColumns(BaseEstimator, TransformerMixin):
    """Bỏ các cột (kèm cột _lag của chúng); dùng cho bộ đặc trưng tuyến tính."""

    def __init__(self, cols=()):
        self.cols = cols

    def fit(self, X, y=None):
        self.columns_ = expand_with_lags(self.cols, X.columns)
        return self

    def transform(self, X):
        return X.drop(columns=[c for c in self.columns_ if c in X])


class StationMonthAnomaly(BaseEstimator, TransformerMixin):
    """(x - mean) / std theo (trạm, tháng trong năm), thống kê học trên dữ liệu fit.

    Cột <c>_lagk là giá trị của tháng t-k nên tra thống kê theo tháng t-k.
    """

    def __init__(self, cols=(), station_col="station", date_col="date"):
        self.cols = cols
        self.station_col = station_col
        self.date_col = date_col

    def _month_ref(self, X, col):
        month = pd.to_datetime(X[self.date_col]).dt.month
        return ((month - 1 - lag_of(col)) % 12 + 1).rename("month_ref")

    def fit(self, X, y=None):
        self.columns_ = expand_with_lags(self.cols, X.columns)
        self.stats_, self.global_ = {}, {}
        for c in self.columns_:
            g = X[c].groupby([X[self.station_col], self._month_ref(X, c)])
            self.stats_[c] = pd.DataFrame({"mean": g.mean(), "std": g.std()})
            self.global_[c] = (X[c].mean(), X[c].std())
        return self

    def transform(self, X):
        X = X.copy()
        for c in self.columns_:
            idx = pd.MultiIndex.from_arrays([X[self.station_col], self._month_ref(X, c)])
            st = self.stats_[c].reindex(idx)
            mean, std = st["mean"].to_numpy(), st["std"].to_numpy()
            g_mean, g_std = self.global_[c]
            miss = ~np.isfinite(mean)
            bad = ~np.isfinite(std) | (std == 0)
            if miss.any() or bad.any():
                log.warning("%s: %d dòng thiếu thống kê (trạm, tháng) -> dùng thống kê chung",
                            c, int((miss | bad).sum()))
            mean = np.where(miss, g_mean, mean)
            std = np.where(bad, g_std, std)
            X[c] = (X[c].to_numpy() - mean) / std
        return X
```

- **Kiểm tra:** `test_anomaly_centered_on_train` và `test_transform_finite` ở bước A10.

A7. Hai bộ đặc trưng và pipeline tiền xử lý

- **Mục tiêu:** một hàm dựng pipeline cho mỗi loại mô hình.
    - **"tree"** (Random Forest, XGBoost): giữ đủ đặc trưng.
    - **"linear"** (Ridge, Logistic): bỏ một vế của các cặp tương quan > 0,9 rồi chuẩn hóa.

- **Các cặp có tương quan > 0,9:**

Cặp	Tương quan
`precip_sum` – `D`	0,99
`q_KR_mean` – `q_TC_mean`	0,99
`sm_0_7` – `sm_7_28`	0,98
`q_KR_anom` – `q_TC_anom`	0,98
`easterly_wind` – `easterly_days`	0,95
`tmax_mean` – `tmax_max`	0,93
`precip_sum` – `rain_days`	0,92
`rain_days` – `dry_spell_max`	0,91
`sm_7_28` – `sm_28_100`	0,90

- **Thứ tự bước:**
  - HotDayFlag → Log → (bỏ cột nếu là linear) → StationMonthAnomaly → ColumnTransformer.
  - ColumnTransformer: one-hot cho `station`; passthrough (tree) hoặc StandardScaler (linear) cho các cột số.
  - Cột `date` có kiểu datetime nên bị `remainder="drop"` loại tự động.
- **Số cột đầu ra kỳ vọng** (tính tay):
  - **tree:** 77 + `has_hot_day` + 13 one-hot = **91**.
  - **linear:** bỏ 18 cột (`precip_sum` ×4, `rain_days`, `q_KR_mean`, `q_KR_min`, `q_KR_anom` ×4, `sm_7_28` ×4, `easterly_days`, `tmax_max`, `q_TC_mean`) → 59 + 1 + 13 = **73**.
- **Ghi chú:** `coastal` là tổ hợp tuyến tính của các cột one-hot trạm. Với Ridge/Logistic có regularization thì không sao, nên giữ cho dễ diễn giải.

Phần trong `preprocess.py`:

```
def build_preprocessor(kind, fc):
    """Pipeline tiền xử lý. kind = 'tree' (RF, XGBoost) hoặc 'linear' (Ridge, Logistic)."""
    if kind not in ("tree", "linear"):
        raise ValueError(f"kind phải là 'tree' hoặc 'linear', nhận {kind!r}")
    steps = [("hot", HotDayFlag()),
             ("log", LogFeatures(fc["log1p_cols"], fc["log_cols"]))]
    if kind == "linear":
        steps.append(("drop", DropColumns(fc["linear_drop"])))
    steps.append(("anom", StationMonthAnomaly(fc["anomaly_cols"])))
    ct = ColumnTransformer(
        [("station", OneHotEncoder(handle_unknown="ignore", sparse_output=False), ["station"]),
         ("num", StandardScaler() if kind == "linear" else "passthrough",
          make_column_selector(dtype_include="number"))],
        remainder="drop", verbose_feature_names_out=False)
    ct.set_output(transform="pandas")
    steps.append(("ct", ct))
    return Pipeline(steps)


def get_feature_names(pre):
    """Tên cột sau biến đổi của một preprocessor đã fit (dùng cho SHAP, web)."""
    return list(pre.named_steps["ct"].get_feature_names_out())
```

- **Cách dùng ở giai đoạn B:**

```
model = Pipeline([("prep", build_preprocessor("tree", fc)), ("model", XGBClassifier(...))])
```

A8. Chia tập

- **Mục tiêu:** giữ đúng thứ tự thời gian. Dùng thẳng cột `split` có sẵn (do `features.build` tạo), không chia lại.

Tập	Giai đoạn	Số dòng
train	1997-04 → 2018-09	3354
gap (bỏ)	2018-10 → 2018-12 và 2021-10 → 2021-12	78
val	2019-01 → 2021-09	429
test	2022-01 → 2025-09	585

Phần trong `preprocess.py`:

```
def split_frames(df):
    """{'train', 'val', 'test'} -> DataFrame; bỏ các dòng 'gap'."""
    return {name: df[df["split"] == name].reset_index(drop=True)
            for name in ("train", "val", "test")}
```

- **Quy tắc dùng tập test:** chỉ chạy **một lần** ở bước B5, sau khi đã chốt mô hình. Ghi ngày giờ chạy vào báo cáo.

A9. Kiểm định chéo theo thời gian (trong train)

- **Mục tiêu:** chọn tham số mà không chạm vào val/test.
- **Cách chia:**
    - Fold năm Y: train = các tháng ≤ tháng 9 năm Y−1; test = 12 tháng năm Y.
    - Khoảng đệm tháng 10–12 năm Y−1 tránh trường hợp nhãn h ≤ 3 chồng sang phần test.
    - Chia **theo ngày**, nên một tháng của cả 13 trạm luôn nằm cùng một phía.

Phần trong `preprocess.py`:

```
def time_series_folds(df, test_years, gap_months=3):
    """Fold mở rộng theo năm trên các dòng split == 'train'.

    Trả về list (train_idx, test_idx) là vị trí dòng (0..len(df)-1) của chính df,
    truyền thẳng vào cv= của RandomizedSearchCV khi fit trên df đó.
    """
    dates = pd.to_datetime(df["date"]).reset_index(drop=True)
    is_train = (df["split"] == "train").to_numpy()
    folds = []
    for year in test_years:
        train_end = pd.Timestamp(year, 1, 1) - pd.DateOffset(months=gap_months)
        tr = np.flatnonzero(is_train & (dates < train_end).to_numpy())
        te = np.flatnonzero(is_train & (dates.dt.year == year).to_numpy())
        if len(tr) and len(te):
            folds.append((tr, te))
    return folds
```

Kích thước fold phải khớp:

Năm kiểm tra	train	test
2010	1950	156
2014	2574	156
2018	3198	117

- **Cách dùng ở giai đoạn B:** fit trên **chỉ tập train**, nên fold tính trên chính DataFrame đó.

```
train = split_frames(data)["train"]
folds = time_series_folds(train, fc["cv_test_years"], fc["cv_gap_months"])
search = RandomizedSearchCV(model, param_distributions, cv=folds, scoring="balanced_accuracy")
search.fit(select_features(train, fc), train["y_cls"])
```

A10. Xuất dữ liệu model-ready và test

- **Mục tiêu:** cố định đầu vào cho giai đoạn B, để giai đoạn B không đọc lại dataset gốc.
- **Đầu ra cho mỗi h:**
  - `data/processed/model_input_h{h}.csv`: `date`, `station`, `split`, 77 đặc trưng **chưa biến đổi**, `y_reg`, `y_cls`, `y_drought`. Dữ liệu được lưu dạng chưa biến đổi vì biến đổi phải fit lại trong từng fold.
  - `models/feature_list_h{h}.json`.

File mới `src/mekong/forecasting/preprocess.py` (toàn văn, gồm cả các hàm ở A4–A9):

```
"""Tầng model-ready: nhãn, chọn đặc trưng chống rò rỉ, pipeline tiền xử lý, fold thời gian.

Chạy:  python -m mekong.forecasting.preprocess [--horizons 1 2 3]
Đầu ra:
  data/processed/model_input_h{h}.csv   date, station, split, 77 đặc trưng thô, y_reg, y_cls, y_drought
  models/feature_list_h{h}.json         danh sách đặc trưng thô / sau biến đổi, cột bị loại, ngưỡng
"""
import argparse
import json
import logging
from datetime import datetime

import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer, make_column_selector
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

from mekong.config import get_path, load_config, setup_logging
from mekong.forecasting.transformers import (DropColumns, HotDayFlag, LogFeatures,
                                             StationMonthAnomaly)
from mekong.quality.freeze import file_sha256

log = logging.getLogger("forecasting.preprocess")


# ---------------------------------------------------------------------- A4. nhãn
def to_class(spei, thresholds):
    """SPEI -> 0 Hạn (<= lo), 1 Bình thường, 2 Ẩm (>= hi)."""
    lo, hi = thresholds
    s = np.asarray(spei, dtype=float)
    return np.select([s <= lo, s >= hi], [0, 2], default=1)


def make_labels(df, h, thresholds, scale=3):
    y = df[f"target_spei{scale}_h{h}"]
    return pd.DataFrame({
        "y_reg": y,
        "y_cls": to_class(y, thresholds),
        "y_drought": df[f"target_drought_h{h}"].astype(int),
    }, index=df.index)


# ---------------------------------------------------------------------- A5. chọn cột
def feature_columns(df, fc):
    """77 cột số được phép làm đặc trưng (đã loại định danh, metadata, rò rỉ, nhãn)."""
    banned = set(fc["id_cols"]) | set(fc["drop_cols"]) | set(fc["leak_cols"])
    return [c for c in df.columns if c not in banned and not c.startswith(("target_", "y_"))]


def select_features(df, fc):
    """Đầu vào của pipeline: date + station (dùng nội bộ) + 77 cột số."""
    return df[["date", "station", *feature_columns(df, fc)]]


# ---------------------------------------------------------------------- A7. pipeline
def build_preprocessor(kind, fc):
    """Pipeline tiền xử lý. kind = 'tree' (RF, XGBoost) hoặc 'linear' (Ridge, Logistic)."""
```

```
    if kind not in ("tree", "linear"):
        raise ValueError(f"kind phải là 'tree' hoặc 'linear', nhận {kind!r}")
    steps = [("hot", HotDayFlag()),
             ("log", LogFeatures(fc["log1p_cols"], fc["log_cols"]))]
    if kind == "linear":
        steps.append(("drop", DropColumns(fc["linear_drop"])))
    steps.append(("anom", StationMonthAnomaly(fc["anomaly_cols"])))
    ct = ColumnTransformer(
        [("station", OneHotEncoder(handle_unknown="ignore", sparse_output=False), ["station"]),
         ("num", StandardScaler() if kind == "linear" else "passthrough",
          make_column_selector(dtype_include="number"))],
        remainder="drop", verbose_feature_names_out=False)
    ct.set_output(transform="pandas")
    steps.append(("ct", ct))
    return Pipeline(steps)


def get_feature_names(pre):
    """Tên cột sau biến đổi của một preprocessor đã fit (dùng cho SHAP, web)."""
    return list(pre.named_steps["ct"].get_feature_names_out())


# ---------------------------------------------------------------- A8, A9. chia tập
def split_frames(df):
    """{'train', 'val', 'test'} -> DataFrame; bỏ các dòng 'gap'."""
    return {name: df[df["split"] == name].reset_index(drop=True)
            for name in ("train", "val", "test")}


def time_series_folds(df, test_years, gap_months=3):
    """Fold mở rộng theo năm trên các dòng split == 'train'.

    Trả về list (train_idx, test_idx) là vị trí dòng (0..len(df)-1) của chính df,
    truyền thẳng vào cv= của RandomizedSearchCV khi fit trên df đó.
    """
    dates = pd.to_datetime(df["date"]).reset_index(drop=True)
    is_train = (df["split"] == "train").to_numpy()
    folds = []
    for year in test_years:
        train_end = pd.Timestamp(year, 1, 1) - pd.DateOffset(months=gap_months)
        tr = np.flatnonzero(is_train & (dates < train_end).to_numpy())
        te = np.flatnonzero(is_train & (dates.dt.year == year).to_numpy())
        if len(tr) and len(te):
            folds.append((tr, te))
    return folds


# ---------------------------------------------------------------- đọc / ghi
def load_ml(cfg):
    return pd.read_csv(get_path(cfg, "processed") / "dataset_monthly_ml.csv", parse_dates=["date"])


def load_model_input(cfg, h):
    """Dùng ở giai đoạn B: đọc model_input_h{h}.csv."""
    return pd.read_csv(get_path(cfg, "processed") / f"model_input_h{h}.csv", parse_dates=["date"])


def main(argv=None):
    ap = argparse.ArgumentParser(description=__doc__.splitlines()[0])
    ap.add_argument("--config", default=None)
    ap.add_argument("--horizons", nargs="*", type=int, help="mặc định: forecasting.horizons")
    args = ap.parse_args(argv)

    cfg = load_config(args.config)
    setup_logging("preprocess", cfg)
    fc = cfg["forecasting"]
    src = get_path(cfg, "processed") / "dataset_monthly_ml.csv"
    df = load_ml(cfg)
    feats = feature_columns(df, fc)
    log.info("%d đặc trưng thô; loại rò rỉ: %s", len(feats), fc["leak_cols"])
    models_dir = get_path(cfg, "models", mkdir=True)
```

```
    for h in args.horizons or fc["horizons"]:
        labels = make_labels(df, h, fc["class_thresholds"], cfg["features"]["target_scale"])
        out = pd.concat([df[fc["id_cols"]], df[feats], labels], axis=1)
        out.to_csv(get_path(cfg, "processed") / f"model_input_h{h}.csv", index=False)
        counts = pd.crosstab(out["split"], out["y_cls"]).loc[["train", "val", "test"]]
        log.info("h%d số mẫu theo lớp (0 Hạn, 1 BT, 2 Ẩm):\n%s", h, counts)

        train = split_frames(out)["train"]
        names = {k: get_feature_names(build_preprocessor(k, fc).fit(select_features(train, fc)))
                 for k in ("tree", "linear")}
        log.info("h%d: tree %d cột, linear %d cột sau biến đổi",
                 h, len(names["tree"]), len(names["linear"]))
        meta = {
            "horizon": h,
            "features_raw": feats,
            "features_tree": names["tree"],
            "features_linear": names["linear"],
            "leak_excluded": fc["leak_cols"],
            "class_thresholds": fc["class_thresholds"],
            "class_names": fc["class_names"],
            "source": src.name,
            "source_sha256": file_sha256(src),
            "created": datetime.now().isoformat(timespec="seconds"),
        }
        path = models_dir / f"feature_list_h{h}.json"
        path.write_text(json.dumps(meta, ensure_ascii=False, indent=2), encoding="utf-8")
        log.info("đã ghi %s", path)


if __name__ == "__main__":
    main()
```

Thêm vào `Makefile`:

```
preprocess: ## dữ liệu model-ready cho h1-h3
    $(RUN) mekong.forecasting.preprocess
```

File mới `tests/test_preprocess.py`. Test đọc dataset thật trong `data/processed/`; nếu chưa có file thì tự bỏ qua.

```
import numpy as np
import pandas as pd
import pytest

from mekong.config import get_path, load_config
from mekong.forecasting.preprocess import (build_preprocessor, feature_columns, make_labels,
                                           select_features, split_frames, time_series_folds)

CFG = load_config()
FC = CFG.get("forecasting", {})
ML = get_path(CFG, "processed") / "dataset_monthly_ml.csv"
pytestmark = pytest.mark.skipif(not ML.exists() or not FC, reason="chưa có dataset/config")


@pytest.fixture(scope="module")
def df():
    data = pd.read_csv(ML, parse_dates=["date"])
    return pd.concat([data, make_labels(data, 1, FC["class_thresholds"])], axis=1)


def test_forecasting_config():
    need = {"horizons", "class_thresholds", "id_cols", "drop_cols", "leak_cols",
            "anomaly_cols", "log1p_cols", "log_cols", "linear_drop", "cv_test_years"}
    assert need <= set(FC)


def test_feature_columns_no_leak(df):
    feats = feature_columns(df, FC)
    assert len(feats) == 77
```

```
    banned = {"oni", "dmi", "date", "split", "station", "lat", "lon", "month"}
    assert not banned & set(feats)
    assert not [c for c in feats if c.startswith(("target_", "y_"))]


def test_label_counts_h1(df):
    counts = pd.crosstab(df["split"], df["y_cls"])
    assert counts.loc["train"].tolist() == [579, 2005, 770]
    assert counts.loc["val"].tolist() == [93, 308, 28]
    assert counts.loc["test"].tolist() == [42, 375, 168]


def test_anomaly_centered_on_train(df):
    train = split_frames(df)["train"]
    Xt = build_preprocessor("tree", FC).fit_transform(select_features(train, FC))
    means = Xt["sm_0_7"].groupby(train["station"]).mean()
    assert np.allclose(means, 0, atol=1e-6)


@pytest.mark.parametrize("kind,n_cols", [("tree", 91), ("linear", 73)])
def test_transform_finite(df, kind, n_cols):
    parts = split_frames(df)
    pre = build_preprocessor(kind, FC).fit(select_features(parts["train"], FC))
    for name in ("train", "val", "test"):
        Xt = pre.transform(select_features(parts[name], FC))
        assert Xt.shape[1] == n_cols
        assert np.isfinite(Xt.to_numpy()).all(), name


def test_lags_look_back_labels_look_forward(df):
    """Cột _lag phải là giá trị QUÁ KHỨ, nhãn phải là giá trị TƯƠNG LAI (chống rò rỉ)."""
    d = df.sort_values(["station", "date"])
    g = d.groupby("station")
    checks = [("precip_sum_lag1", g["precip_sum"].shift(1)),
              ("spei3_lag3", g["spei3"].shift(3)),
              ("oni_lag2", g["oni"].shift(2)),
              ("y_reg", g["spei3"].shift(-1))]      # nhãn h1 = SPEI-3 tháng sau
    for col, expected in checks:
        ok = expected.notna()
        assert np.allclose(d.loc[ok, col], expected[ok]), col


def test_cv_folds(df):
    train = split_frames(df)["train"]
    folds = time_series_folds(train, FC["cv_test_years"], FC["cv_gap_months"])
    assert len(folds) == 9
    assert (len(folds[0][0]), len(folds[0][1])) == (1950, 156)
    dates = train["date"]
    for tr, te in folds:
        assert dates.iloc[tr].max() < dates.iloc[te].min() - pd.DateOffset(months=3)
```

Thứ tự chạy khi đã chép code vào project:

```
make check-raw && make preprocess && make test
ls data/processed/model_input_h*.csv models/feature_list_h*.json
```

- **Kỳ vọng:**
  - Log in đúng bảng số mẫu ở A4.
  - Log báo "tree 91 cột, linear 73 cột".
  - `make test` pass toàn bộ (5 test cũ + 8 test mới: `test_transform_finite` chạy 2 lần cho tree và linear).
- **Nếu số cột lệch:** in `get_feature_names(...)` để tìm cột thừa/thiếu, sửa `linear_drop` trong config.

A11. Chạy lại baseline với dữ liệu sạch

- **Mục tiêu:** có con số baseline tin cậy (không rò rỉ `oni`/`dmi`) để thay bảng ở mục 2.2.
- **Đầu ra:** `reports/week3_baselines.csv`. Chỉ đánh giá trên **val**, chưa chạm test.

File mới `src/mekong/forecasting/evaluate.py` (phần lõi; bước B1 sẽ bổ sung bootstrap và chỉ số theo nhóm):

```
"""Chỉ số đánh giá dùng chung cho mọi mô hình dự báo hạn."""
import numpy as np
from sklearn.metrics import (accuracy_score, balanced_accuracy_score, f1_score,
                             mean_absolute_error, mean_squared_error, recall_score)

DROUGHT = 0   # mã lớp Hạn trong y_cls


def classification_metrics(y_true, y_pred):
    y_true, y_pred = np.asarray(y_true), np.asarray(y_pred)
    d_true, d_pred = y_true == DROUGHT, y_pred == DROUGHT
    return {
        "accuracy": accuracy_score(y_true, y_pred),
        "balanced_accuracy": balanced_accuracy_score(y_true, y_pred),
        "macro_f1": f1_score(y_true, y_pred, average="macro", zero_division=0),
        "drought_recall": recall_score(d_true, d_pred, zero_division=0),
        "drought_f1": f1_score(d_true, d_pred, zero_division=0),
    }


def regression_metrics(y_true, y_pred):
    return {
        "rmse": float(np.sqrt(mean_squared_error(y_true, y_pred))),
        "mae": float(mean_absolute_error(y_true, y_pred)),
    }
```

File mới `src/mekong/forecasting/baseline.py`:

```
"""Baseline Climatology / Persistence / tuyến tính (Logistic + Ridge) trên tập val.

Chạy:  python -m mekong.forecasting.baseline   (sau mekong.forecasting.preprocess)
Đầu ra: reports/week3_baselines.csv
"""
import argparse
import logging

import pandas as pd
from sklearn.linear_model import LogisticRegression, Ridge
from sklearn.pipeline import Pipeline

from mekong.config import get_path, load_config, setup_logging
from mekong.forecasting.evaluate import classification_metrics, regression_metrics
from mekong.forecasting.preprocess import (build_preprocessor, load_model_input, select_features,
                                           split_frames, to_class)

log = logging.getLogger("forecasting.baseline")


def climatology(train, df):
    """Trung bình (hồi quy) và lớp phổ biến nhất (phân loại) theo (trạm, tháng) trên train."""
    keys = [train["station"], train["date"].dt.month.rename("m")]
    reg = train["y_reg"].groupby(keys).mean()
    cls = train["y_cls"].groupby(keys).agg(lambda s: s.mode().iloc[0])
    idx = pd.MultiIndex.from_arrays([df["station"], df["date"].dt.month])
    return reg.reindex(idx).to_numpy(), cls.reindex(idx).to_numpy()


def persistence(df, thresholds, scale):
    """SPEI tương lai = SPEI hiện tại."""
    reg = df[f"spei{scale}"].to_numpy()
    return reg, to_class(reg, thresholds)


def linear(train, df, fc):
```

```
    seed = fc["random_state"]
    clf = Pipeline([("prep", build_preprocessor("linear", fc)),
                    ("model", LogisticRegression(C=0.1, class_weight="balanced",
                                                 max_iter=2000, random_state=seed))])
    reg = Pipeline([("prep", build_preprocessor("linear", fc)), ("model", Ridge(alpha=1.0))])
    X_tr, X = select_features(train, fc), select_features(df, fc)
    clf.fit(X_tr, train["y_cls"])
    reg.fit(X_tr, train["y_reg"])
    return reg.predict(X), clf.predict(X)


def main(argv=None):
    ap = argparse.ArgumentParser(description=__doc__.splitlines()[0])
    ap.add_argument("--config", default=None)
    args = ap.parse_args(argv)

    cfg = load_config(args.config)
    setup_logging("baseline", cfg)
    fc, scale = cfg["forecasting"], cfg["features"]["target_scale"]
    rows = []
    for h in fc["horizons"]:
        parts = split_frames(load_model_input(cfg, h))
        train, val = parts["train"], parts["val"]
        preds = {
            "climatology": climatology(train, val),
            "persistence": persistence(val, fc["class_thresholds"], scale),
            "linear (Ridge/Logistic)": linear(train, val, fc),
        }
        for name, (reg, cls) in preds.items():
            rows.append({"horizon": h, "model": name,
                         **classification_metrics(val["y_cls"], cls),
                         **regression_metrics(val["y_reg"], reg)})
    res = pd.DataFrame(rows).round(3)
    out = get_path(cfg, "reports", mkdir=True) / "week3_baselines.csv"
    res.to_csv(out, index=False)
    print(res.to_string(index=False))
    log.info("đã ghi %s", out)


if __name__ == "__main__":
    main()
```

Thêm vào `Makefile`:

```
baseline:   ## baseline trên val (dữ liệu sạch)
    $(RUN) mekong.forecasting.baseline
```

- **Kiểm tra / kỳ vọng:**
    - `reports/week3_baselines.csv` có 9 dòng (3 mô hình × 3 horizon).
    - Climatology có balanced accuracy khoảng 0,33–0,40 (thấp nhất trong 3 mô hình).
    - Mô hình tuyến tính ở h1 có balanced accuracy ≥ Persistence.
    - Phần chênh lệch so với bảng 2.2 chính là mức ảnh hưởng của rò rỉ `oni`/`dmi` → ghi vào `WEEK3_STEP.md` và cập nhật mục 2.2.

Hoàn thành giai đoạn A khi có:

- `data/MANIFEST.sha256`;
- 3 file `model_input_h*.csv` và 3 file `feature_list_h*.json`;
- `make test` pass;
- `reports/week3_baselines.csv`.

---

4. Giai đoạn B — Mô hình dự báo hạn (Week 3–4)

4.1. Danh sách thuật toán (cố định)

Chiến lược **direct**: mỗi horizon là một mô hình riêng. Mỗi thuật toán chạy ở cả hai bài toán: **hồi quy** (giá trị SPEI-3, dùng để vẽ đồ thị) và **phân loại 3 lớp** (KPI E1, bản đồ).

#	Thuật toán	Hồi quy	Phân loại	Pipeline (A7)
1	Climatology	trung bình SPEI-3 theo (trạm, tháng) trên train	lớp phổ biến nhất theo (trạm, tháng)	không cần
2	Persistence	SPEI-3(t+h) = SPEI-3(t)	lớp của SPEI-3(t)	không cần
3	Mô hình tuyến tính	**Ridge Regression**	**Logistic Regression** đa lớp, `class_weight="balanced"`	linear
4	**Random Forest**	`RandomForestRegressor`	`RandomForestClassifier`, `class_weight="balanced_subsample"`	tree
5	**XGBoost**	`XGBRegressor` + 2 mô hình phân vị (q0,1 và q0,9; `objective="reg:quantileerror"`)	`XGBClassifier`, `sample_weight` cân bằng lớp	tree

Không dùng SMOTE/oversampling: mẫu tổng hợp phá cấu trúc thời gian và tương quan giữa các cột trễ của cùng một trạm.

B1. Module đánh giá (`forecasting/evaluate.py`)

- Hàm `classification_metrics(y_true, y_pred)` → accuracy, balanced accuracy, macro-F1, recall/precision/F1 của lớp Hạn, ma trận nhầm lẫn.
- Hàm `regression_metrics(y_true, y_pred, y_persist)` → RMSE, MAE, R^2, `skill = 1 - RMSE_model / RMSE_persistence`.
- Hàm `bootstrap_ci(y_true, y_pred, metric, n=1000)` → khoảng tin cậy 95%. **Lấy mẫu theo tháng (block bootstrap)**, không theo dòng, vì 13 trạm cùng tháng tương quan với nhau.
- Hàm `by_group(...)` → chỉ số theo trạm, theo vùng (ven biển / nội đồng) và theo mùa (khô / mưa).

B2. Baseline (#1–#3)

- Đã chạy ở A11. Giữ làm mốc so sánh trong mọi bảng kết quả.

B3. Huấn luyện và tinh chỉnh Random Forest, XGBoost (#4, #5)

- Module `python -m mekong.forecasting.train --horizon h --model {rf,xgb,linear}`. Thêm `xgboost` vào `requirements.txt`.
- **Tinh chỉnh:** `RandomizedSearchCV(n_iter=40, cv=time_series_folds(...), scoring="balanced_accuracy")` cho phân loại, `scoring="neg_root_mean_squared_error"` cho hồi quy.
- **Không gian tham số đề xuất:**

Mô hình	Tham số và miền giá trị
Logistic Regression	`C`: log-uniform 1e-3 – 10
Ridge	`alpha`: log-uniform 1e-2 – 100
Random Forest	`n_estimators`: 300–800; `max_depth`: None, 6–20; `min_samples_leaf`: 3–20; `max_features`: sqrt, 0,3, 0,5
XGBoost	`n_estimators`: 200–800; `max_depth`: 3–6; `learning_rate`: 0,02–0,1; `subsample`: 0,7–1; `colsample_bytree`: 0,5–1; `min_child_weight`: 1–10; `reg_lambda`: 1–10

- **Đầu ra:** `reports/week4_cv_results.csv`, gồm điểm kiểm định chéo trung bình ± độ lệch chuẩn cho từng mô hình × horizon × bài toán.

B4. Hiệu chỉnh xác suất và ngưỡng (trên val)

1. Bọc bộ phân loại tốt nhất bằng `CalibratedClassifierCV(method="isotonic", cv="prefit")`, fit trên **val**. Mục

đích: xác suất "Hạn" hiển thị trên bản đồ có ý nghĩa.
2. **Điều chỉnh ngưỡng lớp Hạn:** quy tắc đoán "Hạn" khi P(Hạn) ≥ τ, ngược lại lấy lớp có xác suất cao nhất trong 2 lớp còn lại.
    - Quét τ từ 0,15 đến 0,5 trên val.
    - Chọn τ tối đa balanced accuracy, với điều kiện recall lớp Hạn ≥ 0,6.
    - Lưu τ vào metadata.
3. Kiểm tra độ tin cậy của xác suất bằng reliability diagram và Brier score.

B5. Chọn mô hình và đánh giá test (một lần)

1. Với mỗi h: chọn mô hình có **balanced accuracy trên val cao nhất**. Nếu hai mô hình chênh < 0,01, ưu tiên mô hình có điểm kiểm định chéo ổn định hơn.
2. Fit lại mô hình đã chọn trên **train + val** (giữ nguyên tham số và τ), rồi đánh giá **một lần** trên test.
3. Báo cáo các chỉ số của B1 kèm khoảng tin cậy bootstrap; so sánh với Persistence và Climatology.

B6. Giải thích và lưu mô hình

- **SHAP** (`shap.TreeExplainer`) cho mô hình cây: biểu đồ tổng thể (summary) và giải thích từng dự báo. Thêm `shap` vào requirements.
- **Lưu bằng `joblib`:**
    - `models/drought_cls_h{h}.joblib` (pipeline + mô hình đã hiệu chỉnh);
    - `models/drought_reg_h{h}.joblib`, `models/drought_q10_h{h}.joblib`, `models/drought_q90_h{h}.joblib`.
- **`models/metadata_h{h}.json`:** tên thuật toán, tham số, τ, danh sách đặc trưng, chỉ số val/test, ngày huấn luyện, sha256 của dữ liệu.
- **Dự báo vận hành** (`python -m mekong.forecasting.predict --month YYYY-MM`): đọc dòng tháng t của 13 trạm → `predict_proba` → ghi `data/processed/forecast_<YYYY-MM>.csv`.

B7. Kế hoạch dự phòng nếu h2 chưa đạt 0,60

Làm lần lượt, dừng khi đạt:

1. Thêm đặc trưng xu hướng: `spei3 − spei3_lag1`, `spei1 − spei1_lag1`.
2. Thêm tương tác `month_sin × oni_lag2`, `month_cos × oni_lag2` (ENSO ảnh hưởng theo mùa).
3. Thêm đặc trưng tổng hợp vùng: trung bình SPEI-3 của 13 trạm tại tháng t.
4. Báo cáo trung thực: KPI cam kết đạt ở h1; h2 nêu mức đạt được và phân tích nguyên nhân.

Hoàn thành giai đoạn B khi có:

- `reports/week4_model_report.md`: bảng 5 thuật toán × 3 horizon trên val, kết quả test của mô hình chọn, SHAP, ma trận nhầm lẫn;
- các file mô hình trong `models/`;
- test B: tải mô hình và dự báo 1 tháng thành công.

---

5. Giai đoạn C — Cảnh báo nguy cơ mặn (Week 5)

C1. Thu thập nhãn mặn (song song từ Week 3)

- **Nguồn:** bản tin dự báo / thông báo xâm nhập mặn của Viện Khoa học Thủy lợi miền Nam; báo cáo của Sở NN&MT các tỉnh ven biển.
- **Ưu tiên:** mùa khô (tháng 12 – 5) các năm 2016, 2020, 2024, sau đó các năm khác.
- **Nhập** vào `data/external/salinity_observations_template.csv`. Cột đã có sẵn: date, station_salinity, province_id, river, lat, lon, salinity_max_ppt, intrusion_4ppt_km, source, url.
- **Mục tiêu:** ≥ 150 quan sát.

C2. Xử lý nhãn mặn (`salinity/labels.py`)

1. **Chuẩn hóa đơn vị** về ‰ (g/l). Số liệu clorua mg/l không quy đổi trực tiếp; ghi riêng hoặc bỏ.
2. **Khử trùng lặp:** cùng trạm đo, cùng ngày, khác nguồn → giữ nguồn chính thức (Viện KHTL miền Nam).
3. **Gộp tháng:** độ mặn lớn nhất theo (tỉnh, tháng).
4. **Gán mức:** < 1‰ Thấp, 1–4‰ Trung bình, > 4‰ Cao. Ngưỡng 1‰ và 4‰ khớp với ngưỡng chịu mặn trong `agri_profile.yaml`.
5. **Ghép** vào dataset tháng theo khóa (`station` = `province_id`, `date`). Giữ cột `source` để truy vết.

C3. Chỉ số rủi ro mặn theo luật — SRI (`salinity/sri.py`) — thuật toán #6

Các thành phần đều chuẩn hóa z theo tháng trong năm, **dùng thống kê của train**:

```
SRI(tỉnh, t+h) = hệ_số_vùng × [ 0,4·(−z(q_TC_anom))       # lưu lượng thấp → mặn vào sâu
                              + 0,3·(−SPEI3_dự_báo(t+h))  # hạn tại chỗ
                              + 0,2·z(easterly_wind)       # gió chướng
                              + 0,1·z(tide_amp_max) ]      # triều cường
hệ_số_vùng = 1,0 nếu coastal = 1, 0,5 nếu nội đồng; SRI = 0 ngoài mùa khô (tháng 6–11)
```

- **Trọng số ban đầu** 0,4 / 0,3 / 0,2 / 0,1 là giả định theo vai trò của từng yếu tố. Nếu có nhãn, thay bằng hệ số của Logistic Regression (C4).
- **Gán mức theo phân vị SRI mùa khô trên train:**
    - < phân vị 60: Thấp;
    - phân vị 60 – 85: Trung bình;
    - phân vị 85: Cao.
- **Kiểm tra định tính:** các tháng 2–4 năm 2016 và 2020 tại BT, TV, ST, BL, CM phải ở mức Cao. Nếu không, xem lại trọng số và ghi vào báo cáo.

C4. Mô hình mặn có học (chỉ khi đủ nhãn) — thuật toán #7, #8

Điều kiện	Mô hình	Đánh giá
≥ 150 quan sát	**Logistic Regression**, dự báo P(độ mặn > 4‰), các đặc trưng như C3	leave-one-season-out (mỗi mùa khô làm tập kiểm tra một lần); ROC-AUC, recall mức Cao
≥ 300 quan sát	thêm **Random Forest Classifier** 3 mức	so sánh với Logistic Regression cùng cách đánh giá

Hoàn thành giai đoạn C khi có: `data/processed/salinity_risk.csv` (13 tỉnh × tháng, 2019 → nay), báo cáo kiểm tra định tính, và mô hình mặn nếu đủ nhãn.

---

6. Giai đoạn D — RAG và hệ khuyến nghị (Week 5–6)

6.1. Phương án: hệ khuyến nghị dựa trên tri thức + RAG

- **Không dùng lọc cộng tác:** không có dữ liệu tương tác của người dùng.
- **Không để LLM tự quyết định khuyến nghị:** ngưỡng nông nghiệp phải chính xác, và LLM có thể bịa số liệu.
- **Hai tầng:**
    1. **Tầng luật quyết định khuyến nghị:** từ `agri_profile.yaml`, dự báo hạn và mức mặn, sinh danh sách hành động có điểm ưu tiên. Kết quả tất định, kiểm thử được.
    2. **Tầng RAG giải thích:** truy hồi tài liệu và để LLM viết lời khuyên **chỉ dựa trên tài liệu**, kèm trích dẫn.

D1. Thu thập kho tri thức (song song từ Week 3)

- **Nguồn:** khuyến cáo phòng chống hạn mặn của Bộ NN&MT, công văn của Sở NN&MT các tỉnh, bản tin của Viện KHTL miền Nam, tài liệu kỹ thuật về giống lúa chịu mặn và tưới cây ăn trái.
- **Mục tiêu:** ≥ 60 tài liệu, phủ 13 tỉnh và 6 nhóm cây/con trong `agri_profile.yaml`.
- **Lưu trữ:** file gốc ở `data/knowledge/raw/`; mỗi tài liệu một dòng trong `data/knowledge/manifest.csv` (cột đã có: doc_id, title, issuer, year, doc_type, provinces, topics, language, file, url).

D2. Xử lý tài liệu (`rag/ingest.py`)

1. **Trích văn bản** bằng `pypdf`. Trang không trích được chữ (PDF scan) → đánh dấu `needs_ocr`, xử lý riêng.
2. **Làm sạch:**
    - Chuẩn hóa Unicode về **NFC** (`unicodedata.normalize("NFC", s)`). Đây là lỗi hay gặp: cùng chữ "mặn" nhưng khác mã sẽ không khớp khi tìm.
    - Bỏ header/footer lặp lại trên nhiều trang, số trang, dấu gạch nối cuối dòng.
3. **Chia đoạn** 400–600 token, chồng 80 token, không cắt giữa bảng. Mỗi đoạn gắn metadata: `doc_id`, tỉnh, chủ đề, năm, trang.
4. **Khử trùng lặp** đoạn gần giống nhau (văn bản chỉ đạo thường bị sao chép giữa các tỉnh): độ tương đồng cosine > 0,95 → giữ một đoạn.
5. **Embedding** bằng `BAAI/bge-m3` (sentence-transformers, vector 1024 chiều); lưu vào bảng `doc_chunks` trong PostgreSQL + pgvector.

D3. Truy hồi (`rag/retrieve.py`) — thuật toán #9, #10, #11

```
truy vấn + bộ lọc (tỉnh, chủ đề)
  → BM25 top-20 (rank-bm25; tách từ bằng underthesea)
  → bge-m3 top-20 (pgvector, cosine)
  → Reciprocal Rank Fusion: score = Σ 1/(60 + hạng)
  → cross-encoder BAAI/bge-reranker-v2-m3 → top-5
```

Nếu bộ lọc theo tỉnh trả về ít hơn 3 đoạn, nới bộ lọc sang tài liệu cấp vùng ("ĐBSCL").

D4. Sinh câu trả lời (`rag/generate.py`) — thuật toán #12

- **Model:** `claude-opus-5-5`, gọi qua Anthropic Python SDK (`anthropic`); `output_config.effort = "low"` cho hỏi đáp.
- **Citations:** mỗi đoạn truy hồi gửi dưới dạng document block có `citations: {enabled: true}`. API trả về đoạn trích chính xác (`cited_text`) để web hiển thị nguồn cho từng câu.
- **Prompt caching:** phần cố định (vai trò, quy tắc "chỉ trả lời từ tài liệu", bảng ngưỡng chịu mặn) đặt ở đầu kèm `cache_control`; các đoạn truy hồi thay đổi đặt sau.
- **Fallback khi bị từ chối:** bật `fallbacks: "default"` (beta `server-side-fallback-2026-07-01`); luôn kiểm tra `stop_reason` trước khi đọc nội dung.
- **Không có tài liệu phù hợp** (điểm rerank thấp) → trả lời "chưa có khuyến cáo chính thức", không gọi LLM để suy diễn.
- **Khóa API:** đặt trong `.env` (`ANTHROPIC_API_KEY`), không commit lên git.

D5. Bộ luật khuyến nghị (`services/recommend.py`)

- **Đầu vào:**
    - tỉnh và danh sách cây/con của tỉnh đó (từ `agri_profile.yaml`);
    - tháng; lịch thời vụ;
    - xác suất 3 mức hạn h1–h3 (B6);
    - mức SRI (C3).
- **Mỗi luật** có dạng: điều kiện → hành động + mức ưu tiên + từ khóa truy hồi. Ví dụ:

    - SRI Cao và tỉnh có sầu riêng → "Trữ nước ngọt, dừng tưới khi độ mặn > 0,5‰" (ưu tiên 1).
    - P(Hạn, h2) ≥ 0,5 và tháng 10–11 → "Xuống giống Đông Xuân sớm để né mặn cuối vụ" (ưu tiên 2).
- **Đầu ra:** danh sách hành động đã sắp xếp. Mỗi hành động được gửi qua D3–D4 để lấy phần giải thích có trích dẫn.
- **Test:** mỗi luật có ít nhất 1 test cho trường hợp kích hoạt và 1 test cho trường hợp không kích hoạt.

D6. Đánh giá RAG

- **Bộ kiểm thử:** 40 câu hỏi có đáp án và tài liệu nguồn do nhóm tự soạn. Ví dụ: "Ngưỡng mặn an toàn để tưới sầu riêng?", "Lịch xuống giống né mặn ở Sóc Trăng?". Trong đó có 5 câu hỏi **ngoài kho** để kiểm tra khả năng từ chối.
- **Chỉ số:**
    - Hit@5, MRR (truy hồi);
    - tỉ lệ câu trả lời có trích dẫn;
    - tỉ lệ trích dẫn đúng nguồn (chấm tay);
    - tỉ lệ từ chối đúng với câu hỏi ngoài kho.
- **Đầu ra:** `reports/week6_rag_eval.csv`. Đạt KPI E4.

---

7. Giai đoạn E — Backend và Web (Week 7)

E1. Cơ sở dữ liệu (`storage/`)

PostgreSQL 16 + pgvector, gồm các bảng:

Bảng	Nội dung
`stations`	13 trạm, tọa độ, vùng, tỉnh cũ/mới
`monthly_features`	dữ liệu tháng từ dataset
`forecasts`	`run_month`, `station`, `horizon`, `spei_pred`, `q10`, `q90`, `p_drought`, `p_normal`, `p_wet`, `model_version`
`salinity_risk`	SRI và mức mặn theo tỉnh × tháng
`metrics`	chỉ số mô hình theo horizon
`doc_chunks`	đoạn tài liệu + embedding

Script nạp dữ liệu từ các file CSV đã có.

E2. API (FastAPI, `api/`)

Endpoint	Trả về
`GET /stations`	13 trạm, tọa độ, vùng, tỉnh cũ/mới
`GET /forecast?station=&month=`	SPEI-3 dự báo h1–h3, khoảng q0,1–q0,9, xác suất 3 lớp
`GET /history?station=&from=&to=`	Chuỗi SPEI-1/3/6/12, mưa, ET0, độ ẩm đất, lưu lượng
`GET /salinity-risk?month=`	SRI và mức nguy cơ mặn của 13 tỉnh
`GET /recommendations?station=&crop=`	Danh sách hành động + giải thích có trích dẫn
`POST /chat`	Hỏi đáp RAG tự do
`GET /metrics`, `GET /explain?station=&h=`	Chỉ số mô hình; giá trị SHAP

Schema dùng Pydantic; test bằng `fastapi.testclient.TestClient`.

E3. Giao diện (React + Vite, `src/mekong/web/`)

- **Bản đồ (Leaflet):**
    - GeoJSON 13 tỉnh địa giới cũ, có nút chuyển sang 6 tỉnh/thành sau sáp nhập (trung bình theo cột `province_2025`).
    - Lớp màu theo mức hạn hoặc nguy cơ mặn; thanh trượt horizon 1/2/3 tháng.
    - Bấm vào tỉnh để mở bảng chi tiết.
- **6 đồ thị (Apache ECharts):**
    1. Chuỗi SPEI-3 lịch sử nối với dự báo h1–h3 và dải q0,1–q0,9.
    2. Xác suất 3 lớp hạn.
    3. SHAP của tỉnh đang chọn.
    4. Ma trận nhầm lẫn và balanced accuracy theo horizon.
    5. Dị thường lưu lượng Tân Châu/Kratie.
    6. Đồng hồ SRI và các thành phần đóng góp.
- **Trang Khuyến nghị:** thẻ hành động theo ưu tiên, có nguồn trích dẫn, kèm khung chat RAG.
- **Kiểm tra:** chạy local đủ 3 luồng: xem dự báo, xem nguy cơ mặn, nhận khuyến nghị.

---

8. Giai đoạn F — Deploy và hoàn thiện (Week 8)

F1. Docker Compose

`docker-compose.yml` đã có sẵn, hiện còn rỗng. Cần 3 dịch vụ:

Dịch vụ	Cấu hình
`db`	image `pgvector/pgvector:pg16`
`api`	`python:3.13-slim` chạy uvicorn, nạp mô hình từ `models/`
`web`	build Vite, phục vụ bằng nginx, reverse proxy `/api` sang dịch vụ api

F2. Máy chủ

- 1 VPS Linux 2 vCPU / 4 GB RAM (đủ cho bge-m3 chạy CPU), dùng Caddy để tự cấp HTTPS.
- Dự phòng: Render (web service + PostgreSQL).

F3. CI và job hàng tháng (GitHub Actions)

- **CI:** `pytest` và build image ở mỗi lần push.
- **Job ngày 8 hàng tháng** (ERA5 trễ khoảng 5–7 ngày): `make fetch external validate build preprocess` → `predict` → tính SRI → ghi vào DB.

F4. Thư viện cần thêm vào requirements

`xgboost`, `shap`, `fastapi`, `uvicorn`, `sqlalchemy`, `psycopg[binary]`, `pgvector`, `sentence-transformers`, `rank-bm25`, `underthesea`, `pypdf`, `anthropic`.

F5. Báo cáo và demo

- Báo cáo đồ án: dữ liệu, phương pháp SPEI, so sánh 5 nhóm thuật toán, mặn, RAG, kiến trúc, hạn chế.
- Slide và video demo. Mỗi tuần một file `WEEKn_STEP.md`.

---

9. Lịch theo tuần

Tuần	Ngày	Việc chính	Bàn giao

3	26/10 – 01/11	A0–A11; B1–B2; EDA (`notebooks/01_eda.ipynb`, ≥ 8 biểu đồ); bắt đầu C1, D1	`model_input_h*.csv`, `week3_baselines.csv`, `WEEK3_STEP.md`
4	02/11 – 08/11	B3–B7	Mô hình h1–h3, `week4_model_report.md` (đạt E1)
5	09/11 – 15/11	C2–C4; D2; E1 (DB)	`salinity_risk.csv`, kho tri thức đã index, DB chạy Docker
6	16/11 – 22/11	D3–D6	Khuyến nghị có trích dẫn, `week6_rag_eval.csv` (đạt E4)
7	23/11 – 29/11	E2–E3	Web local đủ 3 luồng (E2)
8	30/11 – 06/12	F1–F5	URL public, CI, báo cáo, demo (E3, tổng hợp E1–E5)

10. Rủi ro và phương án dự phòng

Rủi ro	Khả năng	Ảnh hưởng	Phương án
Kết quả giảm sau khi loại rò rỉ `oni`/`dmi`	Trung bình	KPI E1	Biết sớm ở A11; áp dụng B7 ngay trong Week 4
Horizon 2 tháng chưa đạt 0,60	Trung bình	KPI E1	B7; báo cáo trung thực, KPI cam kết ở h1
Test ít mẫu Hạn (42) → recall dao động	Cao	Độ tin cậy	Báo cáo khoảng tin cậy bootstrap theo tháng (B1)
Không đủ nhãn mặn	Cao	Mô hình mặn ML	SRI theo luật (C3) vẫn chạy; ML mặn là phần mở rộng
Thiếu tài liệu RAG cho một số tỉnh	Trung bình	KPI E4	Nới bộ lọc sang tài liệu cấp vùng (D3)
Open-Meteo giới hạn lượt gọi (429)	Cao	Job hàng tháng	Job chỉ tải 2 tháng gần nhất; crawler hiện tại đã dừng gọn và chạy lại được
Chi phí / khóa LLM	Thấp	Chat RAG	effort thấp, prompt caching, giới hạn lượt chat/ngày, cache khuyến nghị theo tỉnh-tháng
Giai đoạn chuẩn SPEI (1991–2020) chồng val	Thấp	Độ tin cậy báo cáo	Thí nghiệm phụ với `calib_end = 2018`, báo cáo độ chênh
VPS thiếu RAM cho bge-m3	Thấp	Deploy	Tính embedding sẵn lúc nạp kho; dự phòng model `multilingual-e5-small`

11. Tóm tắt sản phẩm cuối

Hạng mục	Sản phẩm
Dữ liệu	Pipeline tái lập được (checksum), dữ liệu model-ready không rò rỉ, 13 tỉnh 1991–nay
Mô hình hạn	5 nhóm thuật toán (Climatology, Persistence, Ridge/Logistic, Random Forest, XGBoost) × 3 horizon. Mô hình triển khai đạt balanced accuracy ≥ 0,60 (h1, h2)
Cảnh báo mặn	SRI theo luật cho 13 tỉnh; Logistic Regression / Random Forest khi đủ nhãn
Khuyến nghị	Luật từ `agri_profile.yaml` + RAG lai (BM25 + bge-m3 + reranker) + Claude Citations
Website	FastAPI + React; bản đồ Leaflet; 6 đồ thị ECharts; Docker Compose trên VPS có HTTPS; CI + job hàng tháng
Tài liệu	WEEK3–WEEK8_STEP.md, các report trong `reports/`, báo cáo đồ án, slide, video demo